

Federated Multi-Agent Reinforcement Learning for Privacy-Preserving and Energy-Aware Resource Management in 6G Edge Networks

Francisco J. E. N. Andong*, Qi Min[†] *[†]Department of Electronics and information Engineering, Northwestern Polytechnical University (NWPU), Xi'an, China
Emails: esonofranciscojavier@mail.nwpu.edu.cn, drqimin@nwpu.edu.cn

Abstract—As sixth-generation (6G) networks move toward ultra-dense, intelligent edge environments, efficient resource management under stringent privacy, mobility, and energy constraints becomes critical. This paper introduces a novel Federated Multi-Agent Reinforcement Learning (Fed-MARL) framework that incorporates cross-layer orchestration of both the MAC layer and application layer for energy-efficient, privacy-preserving, and real-time resource management across heterogeneous edge devices. Each agent uses a Deep Recurrent Q-Network (DRQN) to learn decentralized policies for task offloading, spectrum access, and CPU energy adaptation based on local observations (e.g., queue length, energy, CPU usage, and mobility). To protect privacy, we introduce a secure aggregation protocol based on elliptic-curve Diffie–Hellman key exchange, which ensures accurate model updates without exposing raw data to semi-honest adversaries. We formulate the resource management problem as a partially observable multi-agent Markov decision process (POMMDP) with a multi-objective reward function that jointly optimizes latency, energy efficiency, spectral efficiency, fairness, and reliability under 6G-specific service requirements such as URLLC, eMBB, and mMTC. Simulation results demonstrate that Fed-MARL outperforms centralized MARL and heuristic baselines in task success rate, latency, energy efficiency, and fairness, while ensuring robust privacy protection and scalability in dynamic, resource-constrained 6G edge networks.

Index Terms—Federated Learning, Multi-Agent Reinforcement Learning, Privacy-Preserving, 6G Networks, Energy Efficiency, Task Offloading

I. INTRODUCTION

Sixth-generation (6G) wireless networks are poised to transform communication systems by enabling ultra-dense connectivity, low-latency services, and intelligent edge processing capabilities [1]. These advances are critical for emerging applications such as autonomous driving, augmented reality, and massive Internet of Things (IoT) deployments, each imposing diverse and stringent quality-of-service (QoS) requirements [2], [3]. Efficiently meeting these demands requires decentralized, real-time resource management frameworks capable of operating in highly dynamic, interference-prone, and energy-constrained environments under strict privacy conditions.

Traditional centralized resource management architectures, which depend on global network knowledge for task offloading, spectrum allocation, and computational scheduling, face significant limitations in 6G contexts [4], [5]. These include scalability bottlenecks, latency, communication overhead, and

privacy concerns, particularly when raw user data must be aggregated [6]. Moreover, node mobility and wireless channel variability further degrade the feasibility of centralized control in ultra-dense edge networks [6]. As a result, decentralized intelligence that enables autonomous agents to make context-aware decisions based on local observations has emerged as a critical design requirement [7].

Multi-agent reinforcement learning (MARL) offers a promising paradigm for distributed decision-making, allowing agents to learn cooperative policies through environmental interaction and limited coordination [8], [9]. However, many existing MARL solutions assume centralized training or full observability, which are incompatible with the partial observability, dynamic topologies, and privacy constraints of 6G edge networks [10]–[13]. Additionally, most frameworks focus on optimizing a single metric (e.g., latency or throughput), overlooking essential trade-offs involving fairness, energy efficiency, and spectral utilization [14], [15].

Federated learning (FL) has recently emerged as a privacy-preserving alternative that supports collaborative training through local computation and periodic parameter aggregation, without exposing raw data [16], [17]. Combining FL with MARL yields the federated multi-agent reinforcement learning (Fed-MARL) approach, which facilitates distributed policy learning from partial observations while enabling communication-efficient synchronization [18], [19]. Nonetheless, standard FL is susceptible to gradient inversion and related inference attacks during model update exchanges [20], [21].

To enhance privacy, we propose a Fed-MARL framework that incorporates a secure aggregation protocol using cryptographic pairwise masking and elliptic-curve Diffie–Hellman (ECDH) key exchange. Assuming synchronous updates, this mechanism guarantees that only the aggregated model is revealed, preserving the confidentiality of individual gradients and strengthening privacy in dynamic 6G edge environments [22].

Despite its potential, current MARL approaches remain limited by their assumptions of global observability and centralized coordination [23]. Moreover, many solutions neglect multi-objective optimization, often focusing narrowly on either MAC-layer or application-layer policies [24]–[26], and rarely consider realistic scenarios involving mobility, traffic heterogeneity, and energy constraints [27]. Addressing these limi-

tations requires a scalable, adaptive, and privacy-preserving learning framework that supports decentralized control under partial observability.

In this paper, we propose a novel Fed-MARL framework for cross-layer, multi-objective resource management in 6G edge networks. Our main contributions are summarized as follows:

- We propose a novel Fed-MARL framework that enables decentralized, cross-layer task offloading, spectrum access, and CPU energy adaptation for 6G edge networks, leveraging privacy-preserving federated learning and multi-agent reinforcement learning.
- We introduce a cross-layer coordination mechanism that jointly manages application-layer decisions (e.g., task offloading) and MAC-layer decisions (e.g., spectrum access and energy consumption), addressing dynamic resource constraints and heterogeneous service types in 6G.
- We introduce a novel adaptation of the secure aggregation protocol from Google's framework to federated multi-agent reinforcement learning, enabling model privacy in dynamic edge environments. This adaptation combines elliptic-curve Diffie–Hellman key exchange and symmetric masking, ensuring strong privacy protection under semi-honest adversaries.
- We model the resource management problem as a partially observable multi-agent Markov decision process (POMMDP), incorporating multi-objective optimization to jointly optimize latency, energy efficiency, spectral efficiency, fairness, and reliability under real-world mobility and traffic conditions in 6G networks.
- We demonstrate, through extensive simulations, that Fed-MARL significantly outperforms centralized MARL and heuristic baselines in terms of task success rate, latency, energy efficiency, and fairness, while maintaining strong privacy protections and reducing communication overhead in dynamic, resource-constrained 6G edge environments.

The paper is structured as follows: Section II reviews related work in multi-agent reinforcement learning, federated learning, and wireless network resource management. Section III introduces the system model and formulates the resource coordination problem. Section IV outlines the proposed Fed-MARL framework, including its training process and reward design. Section V presents the simulation setup and performance evaluation, Section VI discusses trade-offs between performance and complexity in 6G. Finally, Section VII concludes the paper and suggests future research directions.

II. RELATED WORK

A. Resource Management in 6G and Edge Networks

Resource management is a foundational challenge in 6G wireless systems, driven by the need for ultra-reliable low-latency communication in ultra-dense, dynamic environments. Gupta et al. [28] highlighted the complexities of spectrum reuse and interference management in device-to-device (D2D) settings, while Kamath [29] discussed orchestration inefficiencies in centralized 5G control architectures. These approaches often assume global network visibility, which is impractical in

decentralized and privacy-sensitive edge environments. Consequently, recent research has shifted toward decentralized and context-aware schemes that better accommodate the mobility, heterogeneity, and privacy constraints expected in 6G.

B. Multi-Agent Reinforcement Learning in Wireless and Edge Environments

To support decentralized intelligence, multi-agent reinforcement learning (MARL) has gained traction as a method for adaptive decision-making. Liu et al. [30] applied MARL for spectrum access and task offloading in IoT-edge systems but relied on simplified settings and limited scalability. Zhang et al. [31] addressed scalability using consensus-based decentralized MARL; however, their methods struggle with partial observability and dynamic network conditions, and they lack privacy and energy-awareness. In contrast, the proposed Fed-MARL framework addresses these limitations by employing Deep Recurrent Q-Networks (DRQN) for partial observability, secure federated aggregation for privacy, and energy-aware decision-making suitable for 6G edge scenarios.

C. Federated Learning for Distributed Decision-Making

Federated learning (FL) enables collaborative model training across devices without exchanging raw data, thus enhancing privacy. Sharma et al. [32] proposed a general FL architecture for edge systems, and Elbir et al. [33] extended FL to vehicular networks. However, most FL-based studies neglect integration with MARL, overlook multi-objective optimization, and often assume static conditions, limiting their applicability to mobile, dynamic 6G environments where learning must be robust to partial information and rapid state changes.

D. Energy-Aware and Cross-Layer Optimization

Energy efficiency is critical in edge networks where devices operate under strict energy budgets. Sada et al. [34] introduced a reinforcement learning-based offloading method balancing energy and latency, while Lu et al. [35] surveyed RL techniques for cross-layer security at the PHY and MAC layers. Yet, many of these efforts focus on isolated layers or objectives and often rely on centralized control. They rarely integrate energy, latency, and reliability into a unified framework capable of operating under full decentralization.

E. Federated Reinforcement Learning for Edge Intelligence

Recent advances have attempted to unify FL and RL to enable decentralized learning with privacy guarantees. The FAuNO framework [36] uses a semi-asynchronous federated actor–critic method to support partial observability in task offloading but focuses on single-layer optimization and lacks robust cryptographic privacy mechanisms. Other efforts, such as those by Zarandi and Tabassum [37] and Noman et al. [38], show promise but often omit fairness, joint optimization, or strong protection against inference attacks. The proposed Fed-MARL framework fills these gaps by integrating DRQN-based temporal modeling, cryptographic aggregation, and multi-layer optimization across latency, energy, spectral efficiency, and fairness.

F. Research Gaps and Motivation

Despite advancements across MARL, FL, and energy-aware computing, most prior works treat these areas in isolation or under idealized assumptions such as centralized coordination, full observability, or static environments. These assumptions are incompatible with real-world 6G edge deployments characterized by mobility, energy constraints, and privacy risks. This paper addresses these limitations by introducing a unified Fed-MARL framework that combines multi-agent learning, secure federated training, partial observability, and multi-objective cross-layer optimization, offering a practical solution for intelligent resource management in next-generation 6G networks.

III. SYSTEM ARCHITECTURE AND PROBLEM FORMULATION

A. System Model

We consider a federated multi-agent wireless edge computing environment reflecting ultra-dense, decentralized, and dynamic 6G conditions. As shown in Fig. 1, the system consists of a set of agents $\mathcal{N} = \{1, 2, \dots, N\}$, each representing an edge node capable of generating and processing computational tasks while contending for a shared wireless spectrum. These agents operate under partial observability, without centralized coordination, and must make real-time decisions across the application, MAC, and CPU layers to manage task execution, communication, and energy consumption.

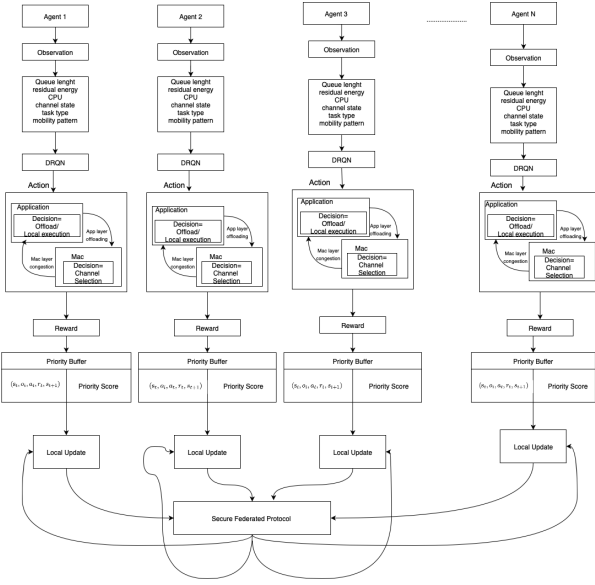


Fig. 1. System Architecture of FERMI-6G.

At each decision interval t , agent i selects an action vector:

$$\mathbf{a}_i(t) = (a_i^{\text{app}}(t), a_i^{\text{mac}}(t), a_i^{\text{cpu}}(t)), \quad (1)$$

where $a_i^{\text{app}}(t) \in \{0, 1\}$ determines whether a task is processed locally or offloaded, $a_i^{\text{mac}}(t) \in \{1, \dots, k\}$ specifies the transmission channel, and $a_i^{\text{cpu}}(t) \in \mathbb{R}$ denotes the CPU frequency for local execution. These actions are tightly coupled and must be jointly optimized for latency, energy

efficiency, and fairness. The system supports multiple traffic types such as Ultra-Reliable Low Latency Communications (URLLC), Enhanced Mobile Broadband (eMBB), and Massive Machine Type Communications (mMTC) with distinct QoS requirements influencing agent behavior.

B. POMMDP-Based Environment Formulation

The learning environment is modeled as a partially observable multi-agent Markov decision process (POMMDP):

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{O}, \mathcal{R}, \gamma \rangle, \quad (2)$$

where $\mathcal{S}$ represents the global state including queue lengths, energy levels, channel states, task types, and mobility. Agent i observes a local state:

$$o_i(t) = \{q_i(t), E_i(t), h_i(t), \lambda_i(t), c_i(t)\}, \quad (3)$$

with queue status $q_i(t)$, residual energy $E_i(t)$, channel gain $h_i(t)$, task type $\lambda_i(t)$, and CPU usage $c_i(t)$. The environment evolves stochastically according to the transition function $\mathcal{T}$, influenced by agent actions and external dynamics such as interference and mobility. Agents learn policies $\pi_i(o_i)$ through reinforcement under uncertainty.

C. Agent Decision and Interaction

Each agent independently learns a policy using a Deep Recurrent Q-Network (DRQN) with LSTM to handle partial observability and temporal dependencies. Observations are mapped to action vectors $\mathbf{a}_i(t)$ for cross-layer decision-making across offloading, channel access, and CPU scaling.

Agents store temporal sequences in a prioritized experience replay buffer for backpropagation through time (BPTT), enhancing stability in non-stationary environments. Training occurs locally, but agents synchronize periodically via secure federated aggregation using elliptic-curve Diffie-Hellman (ECDH) and symmetric AES masking. Pairwise noise masks ensure that the global model is recovered without exposing individual contributions.

D. Multi-Objective Reward Function

The reward function relies on several key variables that capture task performance and system efficiency. Let L denote the task latency, including processing, queuing, and transmission times. D is the task deadline, while R_i measures the reliability of task i as a rolling average of successful completions. Fairness across MAC channels is captured by F using hybrid's index, SE represents the spectral efficiency defined as throughput per unit bandwidth, and EE denotes energy efficiency measured as throughput per unit energy. The MAC success rate is indicated by MAC_{succ} . Each metric is associated with a task-specific weight, namely w_L , w_E , w_F , w_{SE} , w_R , w_{EE} , and w_{MAC} , corresponding to latency, energy, fairness, spectral efficiency, reliability, energy efficiency, and MAC success, respectively.

To ensure comparability across tasks, the key metrics are normalized. Latency tracks the time between task initiation and completion, which is crucial for time-sensitive applications

like URLLC. The normalized latency for task i at time t is defined as

$$\tilde{L}_i(t) = \frac{L_i(t)}{D_i}, \quad (4)$$

where $\tilde{L}_i(t)$ is the normalized latency, $L_i(t)$ is the latency for task i at time t , and D_i is the task deadline. Energy consumed per task, including transmission and computation, is defined as

$$E_i(t) = \frac{\sum_{t=1}^T (E_{\text{tx}}^{(i)}(t) + E_{\text{comp}}^{(i)}(t))}{N_{\text{tasks}}} \quad [\text{J/task}], \quad (5)$$

where $E_i(t)$ is agent i 's energy per task and N_{tasks} is the total number of tasks. This metric encourages agents to reduce energy usage, critical for battery-constrained edge devices. The normalized energy consumption is

$$\tilde{E}_i(t) = \frac{E_i(t)}{E_{\text{max}}}, \quad (6)$$

where $\tilde{E}_i(t)$ is the normalized energy, $E_i(t)$ is the energy consumed by task i at time t , and E_{max} is the maximum energy capacity. Fairness across agents is measured using a combination of Jain's index and channel access entropy. Suppose there are N agents, and let $x_i(t)$ denote the resource allocation (e.g., throughput, successful MAC attempts, or normalized reward) of agent i at time t . Jain's fairness at time t is given by

$$F_{\text{Jain}}(t) = \frac{\left(\sum_{i=1}^N x_i(t)\right)^2}{N \sum_{i=1}^N x_i(t)^2}, \quad F_{\text{Jain}}(t) \in \left[\frac{1}{N}, 1\right]. \quad (7)$$

Suppose each agent i can choose among M channels. Let p_i^c denote the probability that agent i selects channel c over a recent window of T_w time steps. The normalized channel access entropy of agent i is

$$\tilde{H}_i = -\frac{1}{\log_2 M} \sum_{c=1}^M p_i^c \log_2 p_i^c, \quad \tilde{H}_i \in [0, 1], \quad (8)$$

and the average channel access entropy across all agents is

$$H_{\text{avg}} = \frac{1}{N} \sum_{i=1}^N \tilde{H}_i. \quad (9)$$

Finally, the hybrid fairness index used in the reward is

$$F_i(t) = \beta_{\text{jain}} F_{\text{Jain}}(t) + \beta_{\text{entropy}} H_{\text{avg}}, \quad (10)$$

where β_{jain} and β_{entropy} are tunable weights that balance instantaneous fairness and temporal channel diversity. Reliability is measured by the moving average of task success rates over time, providing an indication of the network's long-term stability in completing tasks without failure. The Task Success Rate (TSR) quantifies the percentage of tasks successfully offloaded and completed within deadlines. The reliability of task i is calculated as a rolling average of successes,

$$R_i(t) = \frac{1}{n} \sum_{j=1}^n \text{success}_j, \quad (11)$$

where $R_i(t)$ represents the reliability. The Spectral Efficiency (SE) metric quantifies how effectively data is transmitted per unit of spectrum, which reflects the system's ability to maximize bandwidth usage in dense environments. The normalized spectral efficiency is

$$\tilde{SE}_i(t) = \max(SE_i(t), 0.01), \quad (12)$$

where $\tilde{SE}_i(t)$ is the normalized spectral efficiency. Energy efficiency, the ratio of throughput to energy, is

$$EE_i(t) = \frac{\text{Throughput}_i(t)}{E_{\text{total}}(t)} \quad [\text{bits/J}], \quad (13)$$

capturing the trade-off between performance and energy, motivating agents to maximize throughput while minimizing consumption. Finally, the MAC Success Rate indicates the percentage of successful MAC transmissions, revealing how well the system handles communication despite potential congestion. MAC success rate is

$$\text{MAC}_{\text{rate},i}(t) = \frac{\text{successful MAC attempts}}{\text{total MAC attempts}}, \quad (14)$$

where $\text{MAC}_{\text{rate},i}(t)$ denotes the MAC success rate for task i at time t .

Dynamic penalties are applied to stabilize learning based on latency and energy:

$$P_{\text{dyn}} = \begin{cases} 2, & L_i(t) > 2 \text{ sec}, \\ 1, & \text{otherwise}, \end{cases} \quad (15)$$

$$P_{\text{energy}} = \begin{cases} 2, & \text{remaining energy} < \text{threshold}, \\ 1, & \text{otherwise}. \end{cases} \quad (16)$$

The total reward for agent i at time t is then computed as:

$$\begin{aligned} r_i(t) = & w_L \tilde{L}_i^{-1}(t) P_{\text{dyn}} + w_E \tilde{E}_i^{-1}(t) P_{\text{energy}} \\ & + w_F F_i(t) + w_R R_i(t) \\ & + w_{SE} \tilde{SE}_i(t) + w_{EE} EE_i(t) + w_{MAC} \text{MAC}_{\text{rate},i}(t), \end{aligned} \quad (17)$$

where $\tilde{L}_i^{-1}(t)$ and $\tilde{E}_i^{-1}(t)$ are the inverse normalized latency and energy, respectively, ensuring that lower latency and energy consumption correspond to higher rewards. P_{dyn} and P_{energy} are the dynamic penalties applied based on latency and remaining energy.

To facilitate modular analysis and cross-layer learning, we decompose the total reward into application-layer and MAC-layer components. The application-layer reward, which focuses on computation and latency trade-offs, is given by

$$r_i^{\text{app}}(t) = -\alpha \cdot \tilde{L}_i^{-1}(t) \cdot P_{\text{dyn}} - \beta \cdot \tilde{E}_{\text{comp},i}^{-1}(t) \cdot P_{\text{energy}}, \quad (18)$$

where $\tilde{L}_i^{-1}(t)$ is the inverse normalized latency and $\tilde{E}_{\text{comp},i}^{-1}(t)$ is the inverse normalized energy consumption related to local computation. The MAC-layer reward, which focuses on transmission energy and fairness, is expressed as

$$r_i^{\text{MAC}}(t) = -\beta \cdot \tilde{E}_{\text{trans},i}^{-1}(t) \cdot P_{\text{energy}} + \gamma \cdot F_i(t), \quad (19)$$

where $\tilde{E}_{\text{trans},i}^{-1}(t)$ is the inverse normalized energy consumption related to transmission and $F_i(t)$ represents fairness, captured using the hybrid index. Together, these components allow the reward function to consider both computation and communication aspects of task execution while incorporating dynamic penalties for latency and energy.

The total reward for agent i at time t is the sum of the application layer reward, the MAC layer reward, and additional metrics:

$$r_i(t) = r_i^{\text{app}}(t) + r_i^{\text{MAC}}(t) + \lambda \cdot \Omega_i(t), \quad (20)$$

where $\Omega_i(t)$ represents additional factors such as reliability, spectral efficiency, or other task-specific metrics, and λ is the weight applied to these additional metrics.

This structure allows for layer-specific emphasis during training while promoting end-to-end optimization. It also encourages agents to understand trade-offs between local execution and offloading, as well as how MAC-layer decisions impact both latency and fairness under dynamic network conditions.

E. Federated Learning Protocol

Agents synchronize their Deep Recurrent Q-Network (DRQN) models every K episodes using a secure federated aggregation protocol. After local training, each agent encrypts its model update using symmetric keys derived via Elliptic Curve Diffie-Hellman (ECDH) key exchange, combined with additive masking [21].

Let w_i denote the local model parameters (or update) of agent i , and let m_{ij} be the shared pseudorandom mask generated between agents i and j , derived from their shared symmetric key. The total mask M_i applied by agent i is defined as:

$$M_i = \sum_{j>i} m_{ij} - \sum_{j<i} m_{ji}. \quad (21)$$

The masked model update transmitted by agent i is then:

$$\tilde{w}_i = w_i + M_i. \quad (22)$$

The aggregator computes the average over all masked updates:

$$w = \frac{1}{N} \sum_{i=1}^N \tilde{w}_i = \frac{1}{N} \sum_{i=1}^N (w_i + M_i). \quad (23)$$

Observe that each pairwise mask m_{ij} appears exactly once with a positive sign and once with a negative sign across the sum $\sum_{i=1}^N M_i$, i.e.,

$$\sum_{i=1}^N M_i = \sum_{i=1}^N \left(\sum_{j>i} m_{ij} - \sum_{j<i} m_{ji} \right) = 0. \quad (24)$$

Therefore, the masks cancel out in the aggregate and the final aggregation becomes:

$$w = \frac{1}{N} \sum_{i=1}^N w_i. \quad (25)$$

This protocol ensures privacy of individual updates through masking while allowing the server to compute the exact global average. The aggregated model is broadcast to all agents, providing fairness, consistency, and scalability, with efficient communication and robustness in mobile networks.

F. Optimization Problem

The global objective of Fed-MARL is to learn decentralized policies $\{\pi_i\}_{i=1}^N$ that maximize long-term cumulative rewards under constraints:

$$\begin{aligned} \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_i(t) \right] \\ \text{s.t. } E_i(t) \geq E_i^{\min}, \quad \forall i, t, \\ o_i(t) \subseteq s(t), \quad (\text{Partial Observability}) \\ \text{Communication cost} \leq C_{\max}. \end{aligned} \quad (26)$$

This formulation captures the challenge of learning scalable and privacy-preserving control policies for edge agents in resource-constrained, dynamic 6G environments.

IV. PROPOSED METHOD

A. Overview of the FERMI-6G Framework

We propose FERMI-6G (Federated Energy-aware Reinforcement for Multi-agent Intelligence in 6G), a novel federated multi-agent reinforcement learning (Fed-MARL) framework for decentralized, privacy-preserving, and energy-efficient resource management in ultra-dense 6G edge networks. Building on the system formulation in Section III, FERMI-6G introduces four key innovations:

Temporal Deep Reinforcement Learning: Each agent employs a Deep Recurrent Q-Network (DRQN) with LSTM layers to capture temporal dependencies and manage uncertainty under partial observability (see Section III-C).

Cross-layer Composite Action Space: As defined in Section III-A), agents select a joint action vector

$$a_i(t) = (a_i^{\text{app}}(t), a_i^{\text{MAC}}(t), a_i^{\text{CPU}}(t)), \quad (27)$$

enabling integrated optimization of task offloading, spectrum access, and CPU scaling.

Secure Federated Synchronization: Agents periodically synchronize their models via a privacy-preserving aggregation scheme based on elliptic-curve Diffie-Hellman (ECDH) and AES masking, protecting sensitive updates during coordination (see Section III-E).

Adaptive Multi-Objective Optimization: FERMI-6G employs a dynamic reward function defined in Section III-D, which balances latency, energy consumption, fairness, spectral efficiency, and reliability. The weights of each reward component are adjusted in real time using moving averages of system-level metrics, enabling agents to adapt to diverse traffic types (URLLC, eMBB, mMTC), mobility patterns, and energy constraints.

These integrated mechanisms enable FERMI-6G to operate effectively in multi-agent 6G environments characterized by partial observability, mobility, energy constraints, and dynamic spectrum access.

B. Algorithmic Structure of FERMI-6G

FERMI-6G operates in episodic, synchronous rounds. During each episode, every agent independently interacts with its local environment, maintains its internal state via an LSTM, and updates its DRQN policy using prioritized experience replay. The reward function $r_i(t)$ is computed as defined in Section III-D, combining latency, energy, and fairness components with adaptive weights.

Below is the detailed pseudocode representation of FERMI-6G's federated multi-agent learning workflow:

Algorithm 1 FERMI-6G Sub-Algorithm 1A: Local Agent Learning

Inputs: For each agent $i \in \mathcal{A}$: DRQN policy π_i with parameters θ_i , target network θ_i^{target} , LSTM hidden state h_i , prioritized replay buffer B_i , optimizer, and ECDH key pair. Global parameters: energy threshold $E_{\text{threshold}}$, reward weights (w_L , w_E , w_F , w_{SE} , w_R , w_{EE} , and w_{MAC}), number of episodes N , steps per episode T , and learning parameters ($batch_size$, γ , ϵ , δ , $clip_norm$).

```

1: Initialization:
2: for each agent  $i \in \mathcal{A}$  do
3:   Initialize policy  $\pi_i$  and set  $\theta_i^{\text{target}} \leftarrow \theta_i$ 
4:   Initialize hidden state  $h_i \leftarrow 0$ , replay buffer  $B_i$ , and optimizer
5:   Generate ECDH key pair
6:   Set initial exploration rate  $\epsilon \leftarrow \epsilon_0$ 
7: end for
8: Training Loop:
9: for episode  $n = 1$  to  $N$  do
10:  Reset environment and agent states
11:  for each agent  $i$  do
12:    Set  $h_i(0) \leftarrow 0$ 
13:  end for
14:  for timestep  $t = 1$  to  $T$  do
15:    for each agent  $i$  do
16:      Observe state  $s_i(t) \leftarrow \text{env.get\_state}(i)$ 
17:      Encode features  $\tilde{o}_i(t) \leftarrow \text{observe}(s_i(t))$ 
18:      Update hidden state  $h_i(t) \leftarrow \text{LSTM}(h_i(t-1), \tilde{o}_i(t))$ 
19:      Select action  $a_i(t) \sim \pi_i(h_i(t), \epsilon)$ 
20:      Execute action:  $(s_i(t+1), r_i(t), p_i(t), \text{metrics}) \leftarrow \text{env.step}(i, a_i(t))$ 
21:      Store transition  $(h_i(t), a_i(t), r_i(t), s_i(t+1), p_i(t))$  in  $B_i$ 
22:      Sample minibatch from  $B_i$  and compute TD loss
23:      Clip gradients and update  $\theta_i$ 
24:    end for
25:  end for
26:  Update exploration rate:  $\epsilon \leftarrow \max(\epsilon_{\text{end}}, \epsilon \cdot \delta)$ 
27: end for

```

FERMI-6G Sub-Algorithm 1A captures the local reinforcement learning process at each agent, including state observation, action selection, experience replay, and prioritized updates. These updates are performed independently before global aggregation.

Algorithm 2 FERMI-6G Sub-Algorithm 1B: Federated Aggregation & Reward Adaptation

Inputs: Same as Sub-Algorithm 1A, plus aggregation interval K and reward adaptation flag.

```

1: Secure Aggregation Step (every  $K$  episodes):
2: if  $n \bmod K = 0$  and  $n > 0$  then
3:   for each agent  $i \in \mathcal{A}$  with  $E_i > E_{\text{threshold}}$  do
4:     Generate ECDH key pair (if not existing)
5:     Compute AES masks  $m_{ij}$  and aggregate:  $M_i = \sum_{j>i} m_{ij} - \sum_{j<i} m_{ji}$ 
6:     Mask local model:  $\theta'_i = \theta_i + M_i$ 
7:     Send  $\theta'_i$  to aggregator
8:   end for
9:   Aggregator computes global model:  $\theta_g = \frac{1}{|\mathcal{A}|} \sum_i \theta'_i$ 
10:  Broadcast  $\theta_g$  to all agents
11:  Each agent updates:  $\theta_i \leftarrow \theta_g$ ,  $\theta_i^{\text{target}} \leftarrow \theta_g$ 
12: end if
13: Reward Adaptation:
14: if  $\text{reward\_adaptation\_enabled}$  then
15:  Adjust  $(w_L, w_E, w_F)$  based on system-level thresholds
16:  Normalize reward weights:  $w_j \leftarrow w_j / \sum_k w_k$ 
17: end if
18: Logging:
19: At episode end, record mean reward  $\mathbb{E}[r_i]$ , latency, energy, fairness index, and model divergence

```

Once local training in FERMI-6G Sub-Algorithm 1A is complete, FERMI-6G Sub-Algorithm 1B executes, performing secure federated aggregation of model parameters and optional reward adaptation. This step ensures robustness against energy-constrained agents while optimizing fairness and efficiency. During aggregation, only agents meeting the energy threshold participate, and masking ensures privacy of local updates. Reward adaptation dynamically adjusts system-level weights to balance latency, energy, and fairness, maintaining overall performance across heterogeneous agents. Finally, relevant metrics are logged at the end of each episode to monitor convergence and guide future adaptation.

C. System Integration and Optimizations

Algorithm 1 captures the core FERMI-6G mechanisms, linking each module to system-level components. Each agent uses an LSTM-based DRQN to encode temporal dependencies, producing actions across application, MAC, and CPU layers. These actions influence delay, interference, and energy metrics, which feed into both the reward computation and experience prioritization. Transitions with higher delay or interference are sampled more frequently to improve convergence.

Secure aggregation (Algorithm 2) employs ECDH/AES masking to protect model parameters, and agents below the energy threshold $E_{\text{threshold}}$ are excluded from updates. Reward adaptation (optional) adjusts weights α_j based on moving averages of system metrics to adapt in real time to latency, energy, and fairness variations. Experiments show that using

fixed, pre-defined weights also achieves good performance, so this step can be optionally enabled depending on implementation preference.

D. Key Innovations and Differentiators

Unlike prior works that address individual layers in isolation [18], [39], FERMI-6G integrates privacy-preserving federated synchronization, energy-aware model filtering, and real-time reward adaptation into a single cross-layer learning framework. It jointly optimizes application, MAC, and CPU-layer actions while maintaining reliability under realistic 6G conditions, including dynamic spectrum, non-i.i.d. traffic, and resource-constrained edge devices.

V. EXPERIMENTAL EVALUATION

This section evaluates the performance of the proposed FERMI-6G (Fed-MARL) framework under various network conditions, using quantitative metrics. The evaluation compares FERMI-6G with multiple baselines to assess adaptability, scalability, robustness, and privacy resilience in ultra-dense 6G edge networks. The simulations were conducted on a MacBook Pro (13-inch, 2016), equipped with a 2.9 GHz Intel Core i5 processor, 8 GB of RAM, and Intel Iris Graphics 550.

A. Evaluation Framework and Metrics

To evaluate FERMI-6G's performance, we use a set of key metrics that address various aspects of system effectiveness, including Reliability, Latency, Energy Consumption, Energy Efficiency, Spectral Efficiency, Fairness, MAC Success Rate, Task Completion Time, Offloading Delay, Failure Rate, Throughput, Scalability, and Privacy Protection. Task Completion Time also known as latency for successful tasks, measures the overall time required to process and complete a task, factoring in offloading and computation delays. Offloading Delay specifically tracks the time spent offloading tasks to the edge server, providing insights into network responsiveness. The Failure Rate monitors the percentage of tasks that fail due to issues like congestion or resource unavailability, with a lower failure rate indicating better robustness. Throughput quantifies the total data successfully transmitted over the network, evaluating how efficiently the system utilizes available resources. Scalability assesses the system's ability to handle a growing number of agents or higher network loads without significant performance degradation. Finally, Privacy Protection evaluates the system's capacity to secure model updates through a secure aggregation protocol, ensuring privacy during the offloading process. These metrics collectively inform the Reward Function, which is designed to optimize the trade-off between latency, energy efficiency, fairness, spectral efficiency, and reliability. Together, they provide a comprehensive framework for assessing FERMI-6G's performance under dynamic, real-world conditions.

B. Simulation Setup for Evaluating FERMI-6G in Dense Urban 6G Environments

The simulation setup for evaluating FERMI-6G is designed to replicate the conditions of a dense urban 6G network,

incorporating heterogeneous edge devices such as IoT devices, autonomous vehicles, and base stations, all operating in dynamic environments. The setup integrates multi-agent reinforcement learning (MARL) principles to manage tasks like offloading, energy management, and resource allocation across agents, ensuring that these components are responsive to the challenges of a 6G network.

1) *Network Model and Agent Configuration*: The network consists of a defined number of agents, each representing different devices like IoT sensors or vehicles. These agents are positioned randomly on a 100x100 grid, simulating the dynamic mobility of devices in a busy urban environment. Mobility patterns follow a random walk model, with agent velocities ranging from 0.1 to 1.0 m/s. This randomness mimics the unpredictable movements of devices in urban environments, adding realism to the simulation. This setup allows positions to be updated each timestep to reflect the agents' movements and adapt to the changing network topology. The central resource manager, or edge server, is located at the coordinates [50, 50] on the grid.

2) *Task Types and Offloading*: The tasks performed by the agents are categorized into three types, each with distinct characteristics to reflect real-world communication demands in 6G. URLLC tasks have a small size (1 MB) and a stringent deadline of 2 seconds, eMBB tasks are larger (3 MB) with a 5-second deadline, and mMTC tasks are smaller (0.5 MB) but have the longest deadline of 10 seconds. At the beginning of each simulation episode, agents are randomly assigned one of these task types, and their offloading strategies are influenced by factors such as local energy levels, CPU usage, and queue lengths.

3) *Learning Model*: Each agent is modeled as an independent learning entity, utilizing Deep Recurrent Q-Networks (DRQN) with Long Short-Term Memory (LSTM) layers. This setup allows agents to handle the partial observability of the environment, which is a consequence of the dynamic nature of the agents' mobility, energy consumption, and other variables. The primary objectives of the agents are to manage task offloading to the edge server, optimize energy usage while maintaining battery levels above a 20% threshold, and balance CPU usage to meet task deadlines. Each agent makes decisions based on local state observations, including task queue length, energy levels, CPU usage, channel availability, task type, and mobility speed, all of which impact their task offloading, energy management, and resource allocation strategies.

4) *Medium Access Control (MAC) Layer*: The network employs a medium access control (MAC) layer with a set number of channels, each with an initial bandwidth of 150 GHz and a channel capacity set to 2 per channel. The number of available channels is dynamic, and agents attempt to access these channels for task offloading and communication with the edge server. Channel congestion is modeled by tracking the number of agents using each channel, and retransmission probabilities are introduced to account for failed transmission attempts. The maximum queue length for each agent is set to 10, which can lead to retransmission if tasks fail.

5) *Simulation Parameters*: The simulation runs for 800 episodes with 20 steps per episode, during which agents inter-

act with the environment to improve their task offloading and resource management strategies. Key experiment parameters include a 30% probability of retransmitting failed offloading attempts, a noise standard deviation of 0.05 for modeling random noise during channel gain calculations, and a 2-second transmission delay per attempt. Energy consumption is modeled separately for task computation and transmission, and agents are penalized when their energy falls below the 20% threshold. The simulation setup also incorporates several additional parameters, summarized in the table below:

TABLE I
SIMULATION PARAMETERS FOR THE FERMI-6G FRAMEWORK

Parameter	Description	Value/Range
NUM_AGENTS	Number of agents in the simulation	5
EPISODES	Total number of episodes for training	800
STEPS	Steps per episode	20
HIDDEN	Hidden layer size for agent networks	(2, 3, 3)
BUFFER	Size of the replay buffer	10,000
BATCH	Mini-batch size for training	16
GAMMA	Discount factor for future rewards	0.99
LR	Learning rate for the agent optimizer	1e-3
EPS_START	Initial epsilon value for exploration-exploitation tradeoff	1.0
EPS_END	Final epsilon value	0.05
EPS_DECAY	Decay rate for epsilon value over episodes	0.995
SYNC_FREQ	Frequency of synchronizing target networks	10
SEQUENCE_LENGTH	Length of the sequence in DRQN model	20
CLIP_NORM	Gradient clipping norm to prevent exploding gradients	1.0
SMOOTHING_WINDOW	Window size for moving average smoothing of performance metrics	10

As the simulation progresses, agents continuously update their strategies, refining them to maximize performance in terms of task completion, energy efficiency, and communication reliability.

C. Framework Comparison and Results

The experimental setup includes four frameworks for comparison: FERMI-6G, our proposed decentralized cross-layer approach; Fed-MARL, the same framework without cross-layer optimization; Centralized RL, a standard centralized RL with a single-layer policy; and Centralized RL (Cross-Layer), a centralized RL with full cross-layer optimization. All models were tested in environments that mimic realistic 6G network dynamics; however, due to the complexity of the simulated 6G network, simplified versions of the environment were used for the centralized RL frameworks to allow meaningful training and avoid extremely low rewards (which reached around -147) or zero reliability. Each framework was equipped with a prioritized replay buffer to isolate the effects of replay prioritization from the impact of cross-layer optimization. For clarity, the shorthand notation used throughout this paper is as follows: FERMI-6G refers to Fed-MARL with cross-layer optimization, Fed-MARL refers to Fed-MARL without cross-layer optimization, Centralized RL refers to standard centralized RL, and Centralized RL (Cross-Layer) refers to centralized RL with cross-layer optimization.

FERMI-6G incorporates cross-layer optimization, enabling agents to make decisions across multiple layers (application, MAC, CPU) based on local observations, thereby enhancing performance in complex environments. In contrast, Centralized RL uses Deep Recurrent Q-Networks (DRQN) with global state knowledge, but decisions are made using a simplified, single-layer policy, which limits its ability to handle complex, real-world scenarios. Centralized RL (with Cross-Layer) also uses DRQN but includes full cross-layer optimization,

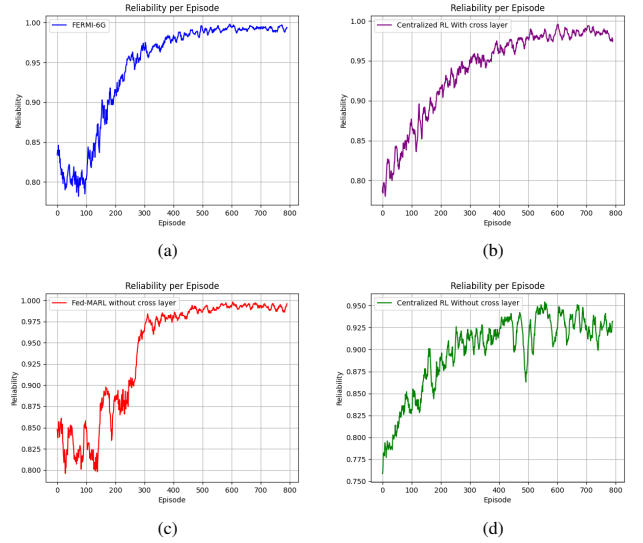


Fig. 2. Reliability comparison across the four frameworks.

enabling global observability and decision-making across all layers. However, it is not realistic for real-world 6G systems due to privacy and communication constraints. To evaluate the impact of cross-layer optimization, we also tested Fed-MARL, which removes the cross-layer optimization while maintaining federated learning and secure aggregation. This version helps isolate the benefits of cross-layer coordination by comparing it to the FERMI-6G framework with cross-layer optimization. To illustrate the impact of the different frameworks on reliability, we present Fig. 2, which shows the reliability performance across all experimental setups. Specifically, Fig. 2(a) shows the reliability of FERMI-6G, Fig. 2(b) shows Centralized RL with cross-layer optimization, Fig. 2(c) represents Fed-MARL, and Fig. 2(d) shows Centralized RL.

a) Baseline Refinement of Fed-MARL: The initial baseline, Fed-MARL, performed poorly, with an average reward of -46 and reliability of only ~30%. This was due to stochastic application-level and MAC-layer actions, particularly the random MAC allocation that introduced high variance and instability. To enable meaningful comparison, we first removed the stochasticity in application and MAC decisions and introduced a deterministic round-robin MAC scheduler, where each channel sequentially grants access to agents. This ensured predictable and equitable channel allocation while preserving other network dynamics (task types, mobility, energy, and constraints). With this modification, Fed-MARL achieved improved reliability (~54%) and more stable latency.

To further strengthen the baseline, we extended Fed-MARL so that each agent learns only application-level actions using a DRQN policy network with a single output head (*app_head*) for offloading decisions. MAC channel selection and CPU allocation are instead handled by fixed heuristics: the least-used channel is selected for MAC access, and CPU usage remains fixed at a default level. The environment continues to simulate full network dynamics (including channel contention, energy consumption, throughput, fairness, latency, reliability, and MAC success) and incorporates these into the reward

signal. Agents interact independently, with no cross-layer coordination, while periodic secure federated aggregation (FedAvg) enables knowledge sharing. With these enhancements, Fed-MARL achieved a reliability of $\sim 94\%$, establishing a robust baseline for fair comparison despite the absence of cross-layer learning.

The results from our experimental setup are summarized in Table II, which compares the four frameworks across several key performance metrics. In this section, we explain these metrics and their implications.

As shown in Fig. 2, the reliability performance of FERMI-6G and Centralized RL with Cross-Layer Optimization diverges notably during the early episodes. FERMI-6G begins with low reliability during episodes 0–100, where the reliability remains flat before gradually improving. This increases exponentially, stabilizing at $96.83\% \pm 4.21\%$ by episode 800. In contrast, Centralized RL with Cross-Layer Optimization starts with a much higher initial reliability and improves more rapidly. It begins increasing almost immediately, stabilizing at $93.81\% \pm 6.13\%$ by episode 400. This rapid early growth contrasts with FERMI-6G’s slower initial progress, but ultimately, FERMI-6G reaches a higher final reliability. The initial delay in FERMI-6G’s performance can be attributed to the time required for cross-layer optimization to adapt and deliver performance gains, whereas Centralized RL with Cross-Layer Optimization benefits from centralized visibility of the entire system from the outset, enabling quicker decisions that impact reliability. Fed-MARL, on the other hand, exhibits substantial reliability instability, fluctuating between 30% and 70% until episode 300. After that, it stabilizes at around $94\% \pm 7.31\%$, offering similar final reliability to Centralized RL with Cross-Layer Optimization but with more early-stage volatility. Centralized RL without cross-layer optimization demonstrates the lowest overall reliability, stabilizing at $89.78\% \pm 5.32\%$, with reliability oscillating between 87.0% and 95.0% during episodes 400–800.

Fig. 3 illustrates the reward dynamics across the frameworks. Centralized RL with Cross-Layer Optimization shows the most stable reward performance, reaching a steady value of 56. The rewards for FERMI-6G, however, are more erratic, with an initial dip to around 10 at episode 0, before recovering and stabilizing at 59.83 ± 6.70 by episode 800. Fed-MARL starts with significant reward instability, fluctuating between 10 and 48 rewards from episode 0 to 280, before stabilizing at 51.51 ± 8.03 after episode 280. This performance is less consistent than the other frameworks and can be attributed to the exploration phase, where the system learns optimal strategies. Centralized RL without cross-layer optimization experiences similar fluctuations, though it increases steadily to 54.31 ± 2.42 rewards in the early episodes, before fluctuating between 51 and 54 rewards in later episodes (after episode 400). This shows that although it benefits from an initial exponential increase, it struggles to stabilize in the long term compared to other frameworks.

Fig. 4 shows the latency performance, which includes both successful and failed task completions. FERMI-6G initially exhibits higher latency ~ 2.8 seconds at episode 100, but this improves exponentially and stabilizes at around 1.12 $\pm$

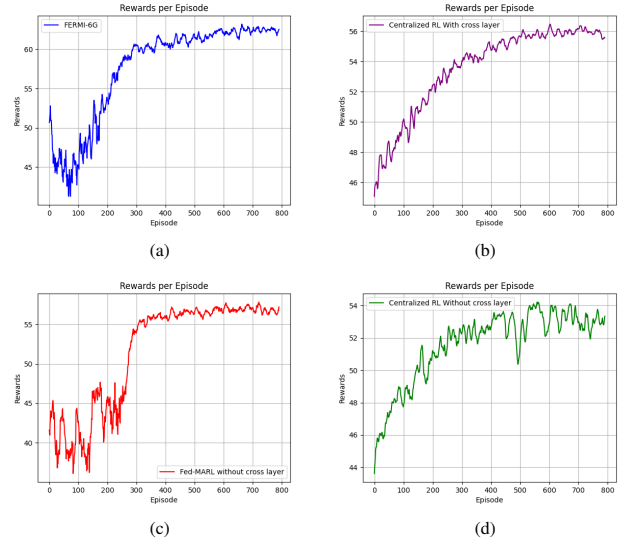


Fig. 3. Rewards comparison across the four frameworks.

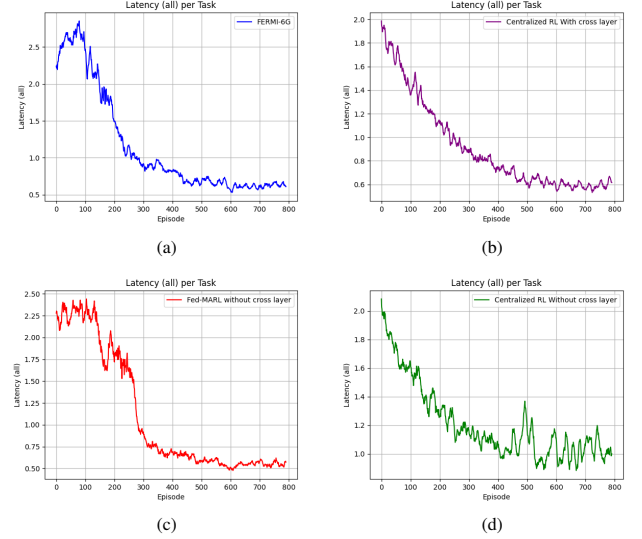


Fig. 4. Latency comparison across the four frameworks.

0.69 seconds by episode 800. Centralized RL with Cross-Layer Optimization experiences the greatest reduction in latency, dropping from 2 seconds to below 0.6 seconds by episode 800. Its final latency is 0.92 ± 0.40 seconds, the lowest among all frameworks. Fed-MARL shows significant fluctuation, starting at 2.5–2.0 seconds in the first 150 episodes, but gradually decreasing to 1.72 seconds by episode 280. After that, it stabilizes but remains higher than FERMI-6G and Centralized RL with Cross-Layer Optimization, with a final value of 0.92 ± 0.53 seconds. Centralized RL without cross-layer optimization follows an erratic pattern. It initially drops quickly to around 0.4 seconds, but from episode 450 onward, the latency fluctuates between 1.4 and 0.4 seconds, ending at 1.36 ± 0.09 seconds by episode 800, the highest of all models.

Fig. 5 illustrates the energy efficiency and consumption across the different frameworks. FERMI-6G leads in energy efficiency with 68.72 ± 19.99 bits/J, outperforming Centralized

TABLE II
FRAMEWORK COMPARISON (MEAN $\pm$ STANDARD DEVIATION)

Metric	Range	FERMI-6G	Centralized RL	Centralized RL (With Cross-Layer)	Fed-MARL
Reward	env-dependent	59.83 $\pm$ 6.70	51.58 $\pm$ 2.42	53.56 $\pm$ 2.97	51.51 $\pm$ 8.03
Reliability	0 $\rightarrow$ 100%	96.83% $\pm$ 4.21%	89.78% $\pm$ 5.32%	93.81% $\pm$ 6.13%	94.00% $\pm$ 7.31%
Energy Efficiency (bits/J)	0 $\rightarrow$ infinite	68.72 $\pm$ 19.99	19.55 $\pm$ 0.71	19.84 $\pm$ 0.74	57.80 $\pm$ 16.37
Energy Consumption (J/task)	0 $\rightarrow$ (energymax=1)	0.0230 $\pm$ 0.0075	0.0663 $\pm$ 0.0038	0.0683 $\pm$ 0.0040	0.0264 $\pm$ 0.0072
Spectral Efficiency (bps/Hz)	0 $\rightarrow$ env-dependent	0.1981 $\pm$ 0.3750	0.5391 $\pm$ 0.0511	0.5642 $\pm$ 0.0575	0.2189 $\pm$ 0.4485
Latency (all attempts, s)	0 $\rightarrow$ infinite	1.12 $\pm$ 0.69	1.36 $\pm$ 0.09	0.92 $\pm$ 0.40	1.11 $\pm$ 0.73
Task Completion Time (s)	0 $\rightarrow$ infinite	0.95 $\pm$ 0.52	1.32 $\pm$ 0.63	0.72 $\pm$ 0.23	0.92 $\pm$ 0.53
Failure Rate (%)	0 $\rightarrow$ infinite	3.17% $\pm$ 4.21%	10.22% $\pm$ 5.32%	6.19% $\pm$ 6.13%	5.99% $\pm$ 7.31%
Privacy Protection	-	High	Low	Low	High
Offloading Delay (s)	0 $\rightarrow$ infinite	3.95 $\pm$ 0.79	0.88 $\pm$ 0.76	0.70 $\pm$ 0.63	3.91 $\pm$ 0.78
Throughput (Gbps)	0 $\rightarrow$ env-dependent	29.20 $\pm$ 2.75	11.35 $\pm$ 0.13	14.11 $\pm$ 0.14	28.26 $\pm$ 2.88
Scalability (Reliability with 50 Agents)	0 $\rightarrow$ 100%	90.01% $\pm$ 0.60%	12.38% $\pm$ 4.92%	20.69% $\pm$ 9.10%	84.05% $\pm$ 5.17%
Fairness	0 $\rightarrow$ 1	0.79 $\pm$ 0.07	0.76 $\pm$ 0.05	0.99 $\pm$ 0.0046	0.17 $\pm$ 0.07
MAC Success Rate (%)	0 $\rightarrow$ 100%	96.83%	99.99%	99.99%	94.00%

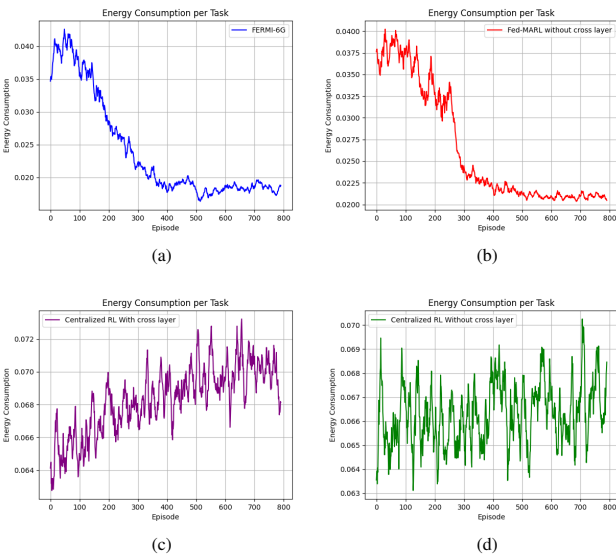


Fig. 5. Energy consumption comparison across the four frameworks.

RL (19.55 $\pm$ 0.71 bits/J) and Centralized RL with Cross-Layer Optimization (19.84 $\pm$ 0.74 bits/J). In terms of energy consumption per task, FERMI-6G uses the least energy, at 0.0230 $\pm$ 0.0075 J/task, followed closely by Fed-MARL (0.0264 $\pm$ 0.0072 J/task). In contrast, both centralized approaches consume significantly more energy (0.0663 $\pm$ 0.0038 J/task for Centralized RL and 0.0683 $\pm$ 0.0040 J/task for Centralized RL with Cross-Layer Optimization), reflecting their higher computational demands. Regarding energy consumption per agent, FERMI-6G starts at around 0.035 energy at episode 0, with fluctuations between 0.037 and 0.039. By episode 150, consumption begins to decrease steadily, reaching 0.011 by episode 800. Fed-MARL exhibits a similar trend but with slightly more fluctuation early on. It stabilizes at a lower value around episode 500, showing a marked reduction in energy consumption compared to earlier stages.

In contrast, both Centralized RL and Centralized RL with Cross-Layer Optimization exhibit unstable energy consumption. While Centralized RL with Cross-Layer maintains a somewhat consistent fluctuation, its overall energy consumption increases over time, with no clear downward trend.

Centralized RL Optimization, despite offering full observability, still struggles with high and erratic energy consumption, lacking the optimization seen in decentralized frameworks like FERMI-6G and Fed-MARL. The absence of effective decentralized decision-making and resource optimization leads to inefficiencies, making these centralized approaches less energy-efficient. As illustrated in Fig. 5(a), FERMI-6G demonstrates a significant reduction in energy consumption over time, from around 0.035 J/task at episode 0 to 0.011 J/task by episode 800, representing a 70% decrease. This decline reflects the framework's ability to optimize resource allocation across the application, MAC, and CPU layers, minimizing redundant computations. Fed-MARL follows a similar, though slightly less efficient, path, stabilizing at a higher energy consumption rate by episode 800.

Beyond reliability, rewards, and latency, the table also includes other metrics such as task completion time, failure rate, privacy protection, throughput, and scalability. For example, FERMI-6G exhibits the fastest task completion time (0.95 $\pm$ 0.52 seconds) and the lowest failure rate (3.17% $\pm$ 4.21%), compared to the other frameworks. These results are consistent with its superior performance in cross-layer optimization, which enables better task scheduling and resource allocation. In terms of scalability, FERMI-6G outperforms all other frameworks, maintaining high reliability (90.01% $\pm$ 0.60%) even when scaled to 50 agents. In contrast, Centralized RL without cross-layer optimization faces severe scalability issues, with reliability dropping to just 12.38% $\pm$ 4.92% as 50 agents are added. Finally, privacy protection is a critical feature of federated learning-based frameworks. FERMI-6G and Fed-MARL maintain high levels of privacy protection, whereas Centralized RL and Centralized RL with Cross-Layer Optimization exhibit lower levels due to their reliance on centralized control, which compromises user data privacy.

VI. DISCUSSION

The comparison reveals key trade-offs between the different frameworks. FERMI-6G consistently outperforms the strengthened Fed-MARL baseline in terms of cumulative reward. Notably, as training episodes increase, FERMI-6G even surpasses the centralized cross-layer framework in application-level performance, highlighting the effectiveness of decentralized cross-layer optimization. However, this performance gain

comes at the expense of fairness, while the centralized cross-layer framework maintains a fairness index of ~ 0.80 , FERMI-6G's fairness drops significantly to 0.3734 ($\approx 37.34\%$). This suggests that for FERMI-6G to maximize performance, agents must adopt more "selfish" policies, prioritizing their own gains over system-wide equity. This trade-off underscores a fundamental tension in decentralized multi-agent systems, while cross-layer coordination enhances efficiency and throughput, it may simultaneously erode fairness across agents. Understanding and mitigating this trade-off is essential for designing practical and equitable 6G systems. The FERMI-6G and centralized RL cross-layer environments differ significantly in terms of realism and modeling detail. FERMI-6G provides a rich per-agent state representation, capturing queue lengths, energy levels, CPU usage, channel conditions with path loss, fading, and blockage, as well as contextual features including task type, size, deadline, mobility, device type, and battery health. MAC access and retransmissions are modeled realistically, accounting for congestion, collisions, and hybrid fairness metrics, with nuanced reward shaping that balances latency, energy, fairness, spectral efficiency, reliability, and MAC success using task-specific weights and dynamic penalties. By contrast, the centralized RL cross-layer environment simplifies the state and MAC modeling, employing limited channel and task context, a probabilistic collision model, and a basic reward function without task-specific differentiation or dynamic penalties. FERMI-6G also incorporates additional mechanisms such as edge caching and smoothed spectral efficiency, absent in the cross-layer model. Consequently, direct quantitative comparisons between the two frameworks may be misleading, as performance differences partly reflect the underlying environment assumptions rather than the learning strategies alone. The spectral efficiency values reported here are normalized to a minimum threshold of 0.01 bps/Hz to ensure consistent performance in challenging environments. While 6G networks are expected to achieve spectral efficiency values in the range of 10-20 bps/Hz in ideal conditions, the lower values in our results reflect the fact that the models used here focus on general performance across varying network conditions. When using the full simulated 6G network, centralized RL frameworks experienced extremely low rewards (around -147) and zero reliability, indicating that standard centralized approaches, even with cross-layer optimization, struggle to handle the complexity of realistic 6G network dynamics. To enable meaningful training, the environments for centralized RL and centralized RL (with cross-layer) were simplified, reducing stochasticity in MAC access, queueing, and task execution. Despite these simplifications, reliability and fairness remained limited, and throughput and spectral efficiency metrics were less stable compared to FERMI-6G. These results emphasize that while centralized cross-layer learning can provide insights under controlled conditions, decentralized cross-layer approaches like FERMI-6G are more robust and effective in complex, realistic 6G scenarios. These findings also highlight the relative advantages of modular heuristics versus learned cross-layer policies. Comparatively, in scenarios with moderate network dynamics, Fed-MARL can perform reasonably well because the heuristics handle MAC

and CPU allocation effectively, while the learned application-level policy still makes competent offloading decisions. However, in highly dynamic or congested networks, FERMI-6G outperforms Fed-MARL, as its learned cross-layer policies can exploit adaptive CPU scaling and channel selection to optimize end-to-end performance. Regarding training cost, Fed-MARL is simpler, faster to train, and more robust to hyperparameter choices, whereas FERMI-6G requires careful tuning to avoid instability. Finally, we note that our training procedure assumes full synchronization every K episodes. While this simplifies analysis, it may not fully reflect practical mobile 6G conditions where agents can be delayed or fail. Importantly, the secure aggregation protocol used in our framework naturally supports partial participation, enabling asynchronous aggregation without compromising security. This capability makes FERMI-6G more amenable to realistic deployment scenarios, where strict synchronization cannot always be guaranteed.

VII. CONCLUSION

In this work, we introduced FERMI-6G, a federated multi-agent reinforcement learning framework for decentralized, privacy-preserving, and energy-aware resource management in ultra-dense 6G edge networks. By combining temporal deep reinforcement learning, a cross-layer composite action space, secure federated synchronization, and adaptive multi-objective optimization, FERMI-6G enables agents to jointly optimize task offloading, spectrum access, and CPU scaling under partial observability and mobility constraints. Experimental results showed that FERMI-6G surpasses existing baselines in reliability, latency, energy efficiency, and scalability while preserving privacy. However, challenges remain in enhancing fairness, reducing latency to meet URLLC requirements, and scaling training efficiency. As future work, we plan to improve spectral efficiency by expanding channel bandwidth, increasing SINR through advanced power control, and integrating technologies such as massive MIMO and refined interference management. We also aim to incorporate ultra-low latency techniques, including edge-native computing, efficient channel access, and integrated air-ground networks, to support demanding applications like immersive AR/VR, industrial IoT, and holographic communication. Finally, we will investigate fairness-aware coordination mechanisms, scalable federated learning strategies, and real-world testbed validation to bring FERMI-6G closer to the ambitious throughput, latency, and reliability targets of next-generation 6G systems.

REFERENCES

- [1] Z. Zhang, Y. Xiao, Z. Ma, M. Xiao, Z. Ding, X. Lei, G. K. Karagiannidis, and P. Fan, "6g wireless networks: Vision, requirements, architecture, and key technologies," *IEEE Vehicular Technology Magazine*, vol. 14, no. 3, pp. 28–41, Sep. 2019.
- [2] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Transactions on Networking*, vol. 24, no. 5, pp. 2795–2808, Oct. 2016.
- [3] H. Guo, X. Zhou, J. Liu, and Y. Zhang, "Vehicular intelligence in 6g: Networking, communications, and computing," *Vehicular Communications*, vol. 33, p. 100399, Jan. 2022.

- [4] I. Ahmad, M. Hoque, J. Huusko, and J. Suomalainen, "Decentralizing 6g security: Existing challenges and future opportunities," in *2024 IEEE Future Networks World Forum (FNWF)*. Dubai, United Arab Emirates: IEEE, 2024.
- [5] J. G. Andrews, S. Buzzi, W. Choi, S. V. Hanly, A. Lozano, A. C. K. Soong, and J. C. Zhang, "What will 5g be?" *IEEE Journal on Selected Areas in Communications*, vol. 32, no. 6, pp. 1065–1082, Jun. 2014.
- [6] E. Li, L. Zeng, Z. Zhou, and X. Chen, "Edge ai: On-demand accelerating deep neural network inference via edge computing," *IEEE Transactions on Wireless Communications*, 2019, accepted for publication. [Online]. Available: <https://doi.org/10.48550/arXiv.1910.05316>.
- [7] M. Chen, Y. Hao, M. Qiu, J. Song, D. Wu, and I. Humar, "Mobility-aware caching and computation offloading in 5g ultra-dense cellular networks," *Sensors*, vol. 16, no. 7, p. 974, Jul. 2016.
- [8] A. K. C. S. Boni, Y. Hablatou, H. Hassan, and K. Drira, "Distributed deep reinforcement learning architecture for task offloading in autonomous iot systems," in *Proc. IEEE International Conference on Communications (ICC)*, 2021, pp. 112–118.
- [9] H. Zhou, K. Jiang, S. He, G. Min, and J. Wu, "Distributed deep multi-agent reinforcement learning for cooperative edge caching in internet-of-vehicles," *IEEE Transactions on Wireless Communications*, vol. 22, no. 12, pp. 9595–9609, Dec. 2023.
- [10] J. Qi, Q. Zhou, L. Lei, and K. Zheng, "Federated reinforcement learning: Techniques, applications, and open challenges," *arXiv preprint arXiv:2108.11887*, Aug. 2021.
- [11] X. Tan, L. Zhou, H. Wang, Y. Sun, H. Zhao, and B.-C. Seet, "Co-operative multi-agent reinforcement-learning-based distributed dynamic spectrum access in cognitive radio networks," *IEEE Internet of Things Journal*, vol. 9, no. 19, pp. 19477–19488, Oct. 2022.
- [12] Q. Wang, J. Huang, and Y. Xu, "Scheduling of real-time wireless flows: A comparative study of centralized and decentralized reinforcement learning approaches," *IEEE/ACM Transactions on Networking*, vol. 32, no. 5, pp. 3699–3714, Oct. 2024.
- [13] A. A. Khan and R. S. Adve, "Centralized & distributed deep reinforcement learning methods for downlink sum-rate optimization," *IEEE Transactions on Wireless Communications*, vol. 19, no. 12, pp. 8410–8426, Dec. 2020.
- [14] Y. S. Nasir and D. Guo, "Multi-agent deep reinforcement learning for dynamic power allocation in wireless networks," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 10, pp. 2239–2250, Oct. 2019.
- [15] S. Cai, J. Fang, and M. Xu, "A multiagent reinforcement learning approach considering fairness for multi-intersection traffic signal control," *IEEE Internet of Things Journal*, vol. 12, no. 13, pp. 23 835–23 851, Jul. 2025.
- [16] Z. Xiong, Y. Zhang, D. Niyato, R. Deng, P. Wang, and L.-C. Wang, "Deep reinforcement learning for mobile 5g and beyond: Fundamentals, applications, and challenges," *IEEE Vehicular Technology Magazine*, vol. 14, no. 2, pp. 44–55, Jun. 2019.
- [17] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.
- [18] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, R. D'Oliveira, R. Eichner, J. H. Lee, D. G. Hsu, S. Jaggi, M. Joshi, J. Kleinberg, T. Konečný, H. B. McMahan, and et al., "Advances and open problems in federated learning," *Foundations and Trends® in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [19] S. Niknam, H. S. Dhillon, and J. H. Reed, "Federated learning for wireless communications: Motivation, opportunities and challenges," *IEEE Communications Magazine*, vol. 58, no. 6, pp. 46–51, Jun. 2020.
- [20] P. R. Ovi and A. Gangopadhyay, "A comprehensive study of gradient inversion attacks in federated learning and baseline defense strategies," in *2023 57th Annual Conference on Information Sciences and Systems (CISS)*, Mar. 2023.
- [21] T. Yuan, H.-M. Chung, and X. Fu, "Pp-marl: Efficient privacy-preserving multi-agent reinforcement learning for cooperative intelligence in communications," *IEEE Network*, vol. 38, no. 5, pp. 196–203, Sep. 2024.
- [22] K. A. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2017.
- [23] E. Eldeeb and H. Alves, "An offline multi-agent reinforcement learning framework for radio resource management," *arXiv preprint arXiv:2501.12991*, Jan. 2025.
- [24] K. Li, S. Ma, P. Dai, J. Ren, and P. Fan, "Wireless resource allocation algorithm based on multi-objective deep reinforcement learning for vehicle-to-vehicle communications," *J. Comput. Res. Dev.*, vol. 61, no. 9, pp. 2229–2245, Sep. 2024.
- [25] G. Thamarasu and R. Sridhar, "Exploring cross-layer techniques for security: Challenges and opportunities in wireless networks," in *Proceedings of MILCOM 2007 - IEEE Military Communications Conference*, Orlando, FL, USA, Oct. 2007.
- [26] B. Wang, L. Liu, and J. Wang, "Multi-agent deep reinforcement learning for task offloading in vehicle edge computing," in *Proceedings of 2023 IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB)*, Jun. 2023.
- [27] Y. Sun, S. Zhou, and J. Xu, "Energy-aware mobility management for mobile edge computing in ultra dense networks," *IEEE Journal on Selected Areas in Communications*, vol. 35, no. 11, pp. 2637–2646, Nov. 2017.
- [28] S. Gupta, R. Patel, R. Gupta, S. Tanwar, and N. Patel, "A survey on resource allocation schemes in device-to-device communication," in *Proceedings of 2022 12th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*, Noida, India, Jan. 2022.
- [29] H. S. Kamath, "5g orchestration and analysis: Dynamic approach, control and challenges faced," in *Proceedings of 2022 IEEE International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE)*, Ballari, India, Apr. 2022.
- [30] X. Liu, J. Yu, Z. Feng, and Y. Gao, "Multi-agent reinforcement learning for resource allocation in iot networks with edge computing," *China Communications*, vol. 17, no. 9, pp. 220–236, Sep. 2020.
- [31] K. Zhang, Z. Yang, H. Liu, T. Zhang, and T. Başar, "Fully decentralized multi-agent reinforcement learning with networked agents," in *Proc. 32nd AAAI Conf. Artif. Intel.*, New Orleans, LA, USA, Feb. 2018, pp. 3695–3702.
- [32] P. Sharma, J. Kashniyal, and E. ESham, "An insight into federated learning: A collaborative approach for machine learning," in *Proc. 2024 4th Int. Conf. on Advancement in Electronics and Communication Engineering (AECE)*, Ghaziabad, India, Nov. 2024.
- [33] A. M. Elbir, B. Soner, S. Çöleri, D. Gündüz, and M. Bennis, "Federated learning in vehicular networks," in *Proc. 2022 IEEE Int. Mediterranean Conf. Commun. Netw. (MeditCom)*, Athens, Greece, Sep. 2022, pp. 72–77.
- [34] A. B. Sada, A. Khelloufi, A. Naouri, H. Ning, and S. Dhelim, "Energy-aware selective inference task offloading for real-time edge computing applications," *IEEE Access*, vol. 12, pp. 72 924–72 937, May 2024.
- [35] X. Lu, L. Xiao, P. Li, X. Ji, C. Xu, S. Yu, and W. Zhuang, "Reinforcement learning-based physical cross-layer security and privacy in 6g," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 425–466, First Quarter 2023.
- [36] F. Metelo, A. Oliveira, S. Racković, C. Soares *et al.*, "Fauno: Semi-asynchronous federated reinforcement learning framework for task offloading in edge systems," *arXiv preprint arXiv:2506.02668*, 2025.
- [37] S. Zarandi and H. Tabassum, "Federated double deep q-learning for joint delay and energy minimization in iot networks," *arXiv preprint arXiv:2104.11320*, 2021.
- [38] H. M. F. Noman, K. Dimiyati, K. A. Noordin, E. Hanafi, and A. Abdrabou, "Fedrl-d2d: Federated deep reinforcement learning-empowered resource allocation scheme for energy efficiency maximization in d2d-assisted 6g networks," *IEEE Access*, vol. 12, pp. 109 775–109 792, Jul. 2024.
- [39] M. Bhutani, B. Lall, and M. Agrawal, "Optical wireless communications: Research challenges for mac layer," *IEEE Access*, vol. 10, pp. 126 969–126 989, Dec. 2022.